

Lab Report No 12
Implementation of Morphological Operations
Digital Image Processing



Submitted By:

[Huzaifa Shahbaz]

[21-CE-009]

[Aqsa Shah]

[21-CE-039]

Submitted to:

Engr. Hareem Khan

Dated:

17th Jan, 2025

Department of Computer Engineering,
HITEC University, Taxila

Task No 1:

Solution:

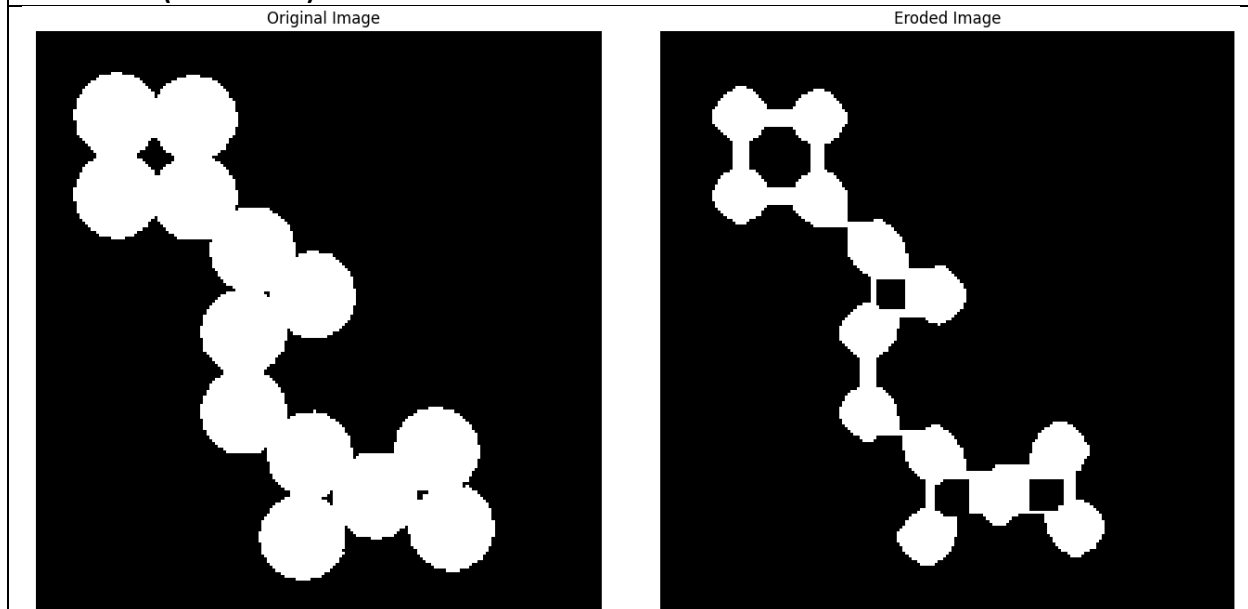
Brief description (3-5 lines)
Dust every particle from every other with Erosion on fig 1.
The code
<pre>import cv2 import numpy as np import matplotlib.pyplot as plt def load_and_process_image(image_path): # Read the image in grayscale img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE) # Convert to binary (if not already) _, binary = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY) return binary def perform_erosion(image, kernel_size=3, iterations=1): # Create kernel for erosion kernel = np.ones((kernel_size, kernel_size), np.uint8) # Perform erosion eroded = cv2.erode(image, kernel, iterations=iterations) return eroded def display_results(original, eroded): # Create figure with two subplots plt.figure(figsize=(10, 5)) # Display original image plt.subplot(121) plt.imshow(original, cmap='gray') plt.title('Original Image') plt.axis('off') # Display eroded image plt.subplot(122) plt.imshow(eroded, cmap='gray') plt.title('Eroded Image') plt.axis('off')</pre>

```

plt.tight_layout()
plt.show()
# Main execution
def main():
    # Load and process the image
    image_path = 'Erosion.jpg' # Replace with your image path
    original = load_and_process_image(image_path)
    # Perform erosion
    # Using kernel_size=5 and iterations=2 to ensure separation
    eroded = perform_erosion(original, kernel_size=5, iterations=2)
    # Display results
    display_results(original, eroded)
if __name__ == "__main__":
    main()

```

The results (Screenshot)



Lab Task No 2:

Solution:

Brief description (3-5 lines)

Carry out erosion on Fig 2 in order to eliminate the noise, followed by dilation to obtain a noise-free image.

The code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
def process_fingerprint(image):
    """
    Process fingerprint image using erosion followed by dilation
    Returns original, eroded and final processed images
    """

    # Convert to grayscale if the image has multiple channels
    if len(image.shape) > 2:
        image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    # Create kernel for morphological operations
    kernel = np.ones((3,3), np.uint8)
    # Perform erosion
    eroded = cv2.erode(image, kernel, iterations=1)
    # Perform dilation on the eroded image
    processed = cv2.dilate(eroded, kernel, iterations=1)
    return image, eroded, processed
def display_results(original, eroded, processed):
    """
    Display the original, eroded, and final processed images side by side
    """

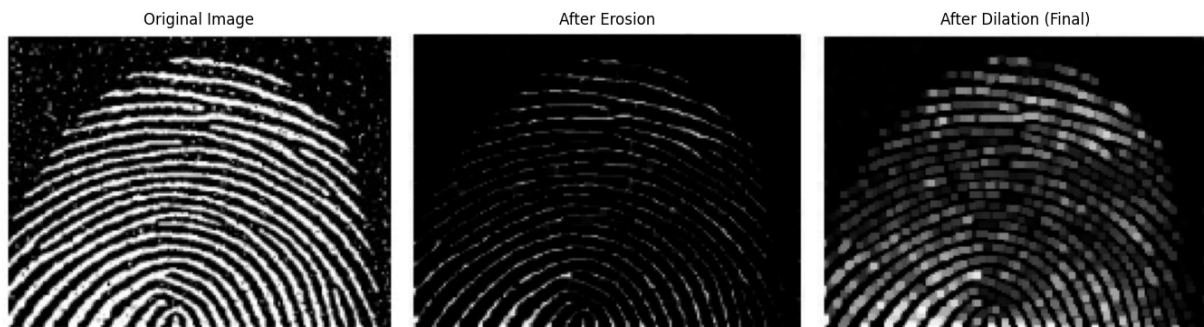
    plt.figure(figsize=(15,5))
    plt.subplot(131)
    plt.imshow(original, cmap='gray')
    plt.title('Original Image')
    plt.axis('off')
    plt.subplot(132)
    plt.imshow(eroded, cmap='gray')
    plt.title('After Erosion')
    plt.axis('off')
    plt.subplot(133)
    plt.imshow(processed, cmap='gray')
    plt.title('After Dilation (Final)')
```

```

plt.axis('off')
plt.tight_layout()
plt.show()
# Main execution
if __name__ == "__main__":
    # Read the image
    image = cv2.imread('fingerprint.jpg')
    # Process the image
    original, eroded, processed = process_fingerprint(image)
    # Display results
    display_results(original, eroded, processed)
    # Save the processed image
    cv2.imwrite('processed_fingerprint.jpg', processed)

```

The results (Screenshot)



Lab Task No 3:

Solution:

Brief description (3-5 lines)

Carry out dilation and erosion processes of a 3x3 dimension on grayscale images for Fig 3. Also compute morphological gradient along with Top-hat operation.

The code

```

import cv2
import numpy as np
import matplotlib.pyplot as plt
def perform_morphological_operations(image_path):

```

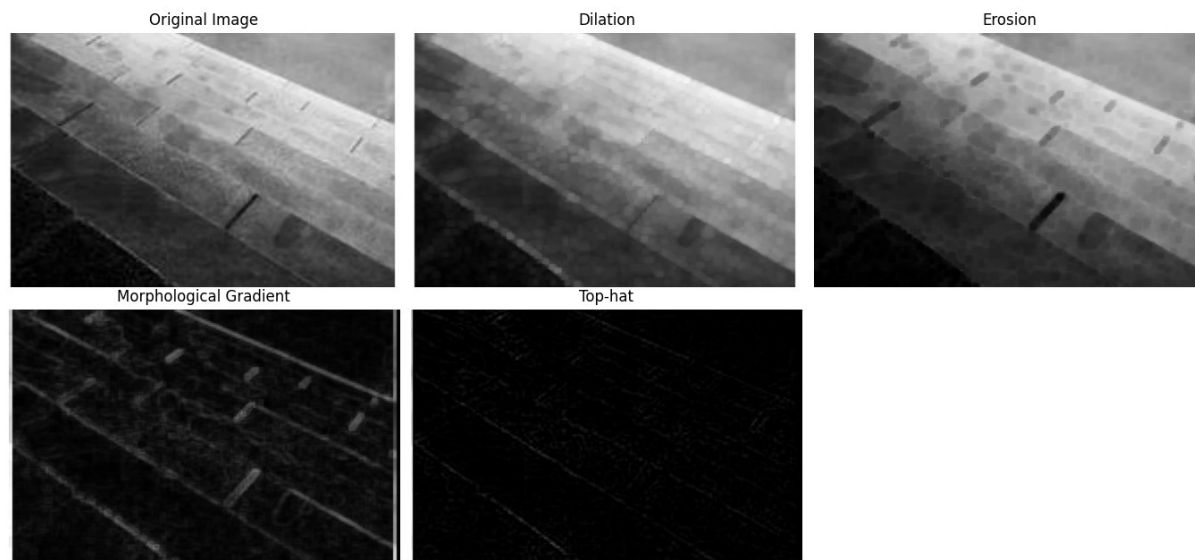
```

# Read the image in grayscale
img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
# Create a 3x3 kernel
kernel = np.ones((3,3), np.uint8)
# Perform dilation
dilation = cv2.dilate(img, kernel, iterations=1)
# Perform erosion
erosion = cv2.erode(img, kernel, iterations=1)
# Morphological gradient (dilation - erosion)
morph_gradient = cv2.subtract(dilation, erosion)
# Top-hat operation (original - opening)
opening = cv2.morphologyEx(img, cv2.MORPH_OPEN, kernel)
tophat = cv2.subtract(img, opening)
# Create subplots to display results
plt.figure(figsize=(15, 10))
plt.subplot(2, 3, 1)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis('off')
plt.subplot(2, 3, 2)
plt.imshow(dilation, cmap='gray')
plt.title('Dilation')
plt.axis('off')
plt.subplot(2, 3, 3)
plt.imshow(erosion, cmap='gray')
plt.title('Erosion')
plt.axis('off')
plt.subplot(2, 3, 4)
plt.imshow(morph_gradient, cmap='gray')
plt.title('Morphological Gradient')
plt.axis('off')
plt.subplot(2, 3, 5)
plt.imshow(tophat, cmap='gray')
plt.title('Top-hat')
plt.axis('off')

```

```
plt.tight_layout()
plt.show()
return dilation, erosion, morph_gradient, tophat
# To use this function:
image_path = 'Grayscale Morphological Operations.jpg'
dilation, erosion, gradient, tophat =
perform_morphological_operations(image_path)
```

The results (Screenshot)



Conclusion

In conclusion, morphological operations like dilation, erosion, opening, and closing are crucial for enhancing and analyzing image structures, providing benefits in tasks like noise reduction, edge detection, and object separation, thus improving the quality and interpretability of image data in computer vision applications.